

AI-Assisted Coding Lab Assignment - 19.1

Name: A. Sushmitha

Roll No: 2303A51541

Batch: 13

Lab 19 – Code Translation: Converting Between Programming Languages

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

Expected Output:

- Equivalent working Java class translated from Python.

Prompt:

Convert the given Python class into equivalent Java code with proper data types, constructor, and methods. Ensure the Java version includes a `main()` method and follows correct syntax. Maintain the same functionality and output as the original Python program.

Python Code:

```
task1.py ×
LAB-19 > task1.py > ...
1  class Student:
2      def __init__(self, name, marks):
3          self.name = name
4          self.marks = marks
5
6      def display(self):
7          print("Name:", self.name)
8          print("Marks:", self.marks)
9
10 # Create object
11 s1 = Student("Sushmitha", 85)
12 s1.display()
13
14
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-19/task1.py"
• Name: Sushmitha
  Marks: 85
○ PS C:\AI Assisted Coding\LAB> 
```

Java Code:

```
LAB-19 > J task1.java > Student
1  class Student {
2      String name;
3      int marks;
4
5      // Constructor
6      Student(String name, int marks) {
7          this.name = name;
8          this.marks = marks;
9      }
10
11     // Method
12     void display() {
13         System.out.println("Name: " + name);
14         System.out.println("Marks: " + marks);
15     }
16
17     // Main method
18     public static void main(String[] args) {
19         Student s1 = new Student(name: "Sushmitha", marks: 85);
20         s1.display();
21     }
22 }
```

Output:

```
PS C:\AI Assisted Coding\LAB> & 'C:\Program Files\Java\jdk-22\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages'
5234c875bee355db6eaa0ec6e7\redhat.java\jdt_ws\LAB_f9513060\bin' 'Student'
Name: Sushmitha
Marks: 85
PS C:\AI Assisted Coding\LAB> 
```

Observation

- The Python class was successfully converted into Java using AI assistance.
- Differences were observed in syntax: Python uses dynamic typing, while Java requires explicit data types.
- The constructor (`__init__`) in Python was correctly mapped to a Java constructor.
- Output from both programs was the same, confirming functional equivalence.

Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

Prompt:

Convert the following recursive C++ factorial function into Python.

Ensure proper indentation, Pythonic syntax, and include a main section.

Maintain the same logic and output.

C++ Code:

```
factorial.cpp X
factorial.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int factorial(int n) {
5      if (n == 0 || n == 1)
6          return 1;
7      else
8          return n * factorial(n - 1);
9  }
10
11 int main() {
12     printf("Program started\n");
13     int num = 5;
14     printf("Factorial: %d\n", factorial(num));
15     return 0;
16 }
```

Output:

```
Program started
Factorial: 120
PS C:\AI Assisted Coding\LAB\LAB-19>
```

Python Code:

```
1 def factorial(n):
2     if n == 0 or n == 1:
3         return 1
4     else:
5         return n * factorial(n - 1)
6
7 if __name__ == "__main__":
8     num = 5
9     print("Factorial:", factorial(num))
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-19> & "C:\Program Files\Python314\python.exe"
Factorial: 120
PS C:\AI Assisted Coding\LAB\LAB-19> & "C:\Program Files\Python314\python.exe"
Factorial: 720
PS C:\AI Assisted Coding\LAB\LAB-19> █
```

Observation:

- The recursive factorial function written in C++ was successfully converted into Python using AI assistance.
- The Python version follows proper indentation and Pythonic syntax, making it more concise and readable.
- The logic of recursion remained unchanged during conversion.
- The program was tested with sample inputs, and both C++ and Python versions produced the same output.
- This confirms that the Python implementation is functionally equivalent to the original C++ code.

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.

- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

Prompt:

Convert the following SQL query into an equivalent Pandas DataFrame operation in Python. Use proper Pandas syntax and assume a DataFrame named employees. Return only the required columns and apply the condition correctly.

SQL:

```
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

Python Code:

```
2
3 import pandas as pd
4
5 # Create sample DataFrame
6 data = {
7     'name': ['Alice', 'Bob', 'Charlie', 'David'],
8     'salary': [40000, 60000, 55000, 30000]
9 }
10
11 employees = pd.DataFrame(data)
12
13 # Pandas equivalent of SQL query
14 result = employees[employees['salary'] > 50000][['name', 'salary']]
15
16 print(result)
```

Output:

```
●      name  salary
1      Bob   60000
2  Charlie   55000
○ PS C:\AI Assisted Coding\LAB\LAB-19> █
```

Observation

- The SQL query was successfully translated into an equivalent Pandas DataFrame operation.
- Initial errors (incorrect import and syntax issues) were identified and corrected.
- The Pandas code correctly filtered rows using the condition (**salary > 50000**) and selected the required columns.
- The output matched the expected result of the SQL query.
- This demonstrates that Pandas can effectively perform SQL-like operations in Python for data analysis.

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will choose a real-time use case where they need code in multiple languages. Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.
- Translating a Java method into C# for enterprise applications.

Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages

Prompt:

Convert the following Python code into equivalent JavaScript. Use appropriate libraries (like fetch or axios) and maintain the same functionality and output. Ensure the code is suitable for browser or Node.js execution.

Python Code:

```
1  import requests
2
3  url = "https://jsonplaceholder.typicode.com/posts/1"
4  response = requests.get(url)
5
6  data = response.json()
7  print("Title:", data['title'])
```

Javascript:

```
1  fetch("https://jsonplaceholder.typicode.com/posts/1")
2    .then(response => response.json())
3    .then(data => {
4      console.log("Title:", data.title);
5    })
6    .catch(error => console.log(error));
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-19> node task4.js
Title: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
PS C:\AI Assisted Coding\LAB\LAB-19> 
```

Observation

- The Python code was successfully translated into JavaScript using AI assistance.
- Initial errors occurred due to missing libraries (requests) and incorrect file naming, which were resolved.
- Both Python and JavaScript programs were executed and produced the same output.
- Differences were observed in syntax and execution: Python uses synchronous-style code, while JavaScript uses asynchronous methods (fetch with .then()).
- This task demonstrated how similar functionality can be implemented across different programming languages with appropriate adjustments.